



Using the SQL Gateway Programmatically

Version 2026.1
2026-04-20

InterSystems®, HealthShare Care Community®, HealthShare Unified Care Record®, InterSystems IRIS® IntegratedML®, InterSystems Caché®, InterSystems Ensemble®, InterSystems HealthShare®, InterSystems IRIS®, InterSystems IRIS® for Health, and TrakCare are registered trademarks of InterSystems Corporation. HealthShare® Health Connect Cloud™, InterSystems® Data Fabric Studio™, and InterSystems Supply Chain Orchestrator™ are trademarks of InterSystems Corporation. TrakCare is a registered trademark in Australia and the European Union.

All other brand or product names used herein are trademarks or registered trademarks of their respective companies or organizations.

This document contains trade secret and confidential information which is the property of InterSystems Corporation, One Congress Street, Boston, MA 02114, or its affiliates, and is furnished for the sole purpose of the operation and maintenance of the products of InterSystems Corporation. No part of this publication is to be used for any other purpose, and this publication is not to be reproduced, copied, disclosed, transmitted, stored in a retrieval system or translated into any human or computer language, in any form, by any means, in whole or in part, without the express prior written consent of InterSystems Corporation.

The copying, use and disposition of this document and the software programs described herein is prohibited except to the limited extent set forth in the standard software license agreement(s) of InterSystems Corporation covering such programs and related documentation. InterSystems Corporation makes no representations and warranties concerning such software programs other than those set forth in such standard software license agreement(s). In addition, the liability of InterSystems Corporation for any losses or damages relating to or arising out of the use of such software programs is limited in the manner set forth in such standard software license agreement(s).

THE FOREGOING IS A GENERAL SUMMARY OF THE RESTRICTIONS AND LIMITATIONS IMPOSED BY INTERSYSTEMS CORPORATION ON THE USE OF, AND LIABILITY ARISING FROM, ITS COMPUTER SOFTWARE. FOR COMPLETE INFORMATION REFERENCE SHOULD BE MADE TO THE STANDARD SOFTWARE LICENSE AGREEMENT(S) OF INTERSYSTEMS CORPORATION, COPIES OF WHICH WILL BE MADE AVAILABLE UPON REQUEST.

InterSystems Corporation disclaims responsibility for errors which may appear in this document, and it reserves the right, in its sole discretion and without notice, to make substitutions and modifications in the products and practices described in this document.

For Support questions about any InterSystems products, contact:

InterSystems Worldwide Response Center (WRC)
Tel: +1-617-621-0700
Tel: +44 (0) 844 854 2917
Email: support@InterSystems.com

Table of Contents

Using the SQL Gateway Programmatically	1
1 FetchSamples Example	1
2 Creating and Using an External Data Set	2
3 Calling ODBC Functions Directly	3
4 Quick Reference for %SQLGatewayConnection	4
4.1 Overview of the %SQLGatewayConnection API	4
4.2 %SQLGatewayConnection Methods and Properties	5
4.3 Supported ODBC Function Calls	8
 List of Tables	
Table 1: Calling ODBC Functions from %SQLGatewayConnection	8

Using the SQL Gateway Programmatically

Important: This article is intended only for developers who need to maintain existing legacy code. When developing new code to access the ODBC SQL Gateway programmatically, InterSystems strongly recommends use of the XDBC Gateway, which provides a much simpler and more maintainable way to accomplish the same tasks.

Note: This article assumes that you have significant experience using ODBC API calls — it is not intended to provide details on how to use ODBC functions. If you encounter any problems, you can monitor the SQL Gateway by enabling logging for both InterSystems IRIS and ODBC (see the “Logging and Environment Variables” chapter in *Using the InterSystems ODBC Driver*).

If you require options that are not provided by the SQL Gateway wizards, you can use the %Library.SQLGatewayConnection class to call ODBC functions from ObjectScript. You can either execute a dynamic query (obtaining a result set) or you can perform low-level ODBC programming. The following topics are discussed in this chapter:

- [FetchSamples Example](#) — lists a simple program that opens a connection, runs a query, and accesses the result set.
- [Creating and Using an External Data Set](#) — demonstrates using %SQL.Statement methods to run queries and access data sets.
- [Calling ODBC Functions Directly](#) — demonstrates how to call ODBC query functions directly, rather than through %SQL.Statement.
- [Quick Reference for %SQLGatewayConnection](#) — provides details about the supported methods and properties.

In the rest of this chapter, %Library.SQLGatewayConnection is referred to by its abbreviated name, %SQLGatewayConnection.

1 FetchSamples Example

The following example provides a simple demonstration of how to open a connection, prepare and execute a query, and access the resulting data set. See the entries in “[Quick Reference for %SQLGatewayConnection](#)” for information on [Connect\(\)](#), [Disconnect\(\)](#), [ConnectionHandle](#), and [sqlcode](#). See the Quick Reference section on “[Supported ODBC Function Calls](#)” for a list of supported ODBC functions and the %SQLGatewayConnection methods that call them.

ClassMethod FetchSamples

Class Member

```
ClassMethod FetchSamples()
{
    #include %occInclude
    //Create new SQL Gateway connection object
    set gc=##class(%SQLGatewayConnection).%New()
    if gc=$$NULLOREF quit $$$ERROR($$$GeneralError,"Cannot create %SQLGatewayConnection.")

    //Make connection to target DSN
    set pDSN="Cache Samples"
    set usr="_system"
    set pwd="SYS"
    set sc=gc.Connect(pDSN,usr,pwd,0)
    if $$$ISERR(sc) quit sc
    if gc.ConnectionHandle="" quit $$$ERROR($$$GeneralError,"Connection failed")

    set sc=gc.AllocateStatement(.hstmt)
    if $$$ISERR(sc) quit sc
}
```

```
//Prepare statement for execution
set pQuery= "select * from Sample.Person"
set sc=gc.Prepare(hstmt,pQuery)
if $$$ISERR(sc) quit sc
//Execute statement
set sc=gc.Execute(hstmt)
if $$$ISERR(sc) quit sc
//Get list of columns returned by query
set sc=gc.DescribeColumns(hstmt, .columnlist)
if $$$ISERR(sc) quit sc

//display column headers delimited by ":"
set numcols=$listlength(columnlist)-1 //get number of columns
for colnum=2:1:numcols+1 {
    write $listget($listget(columnlist,colnum),1),":"
}
write !

//Return first 200 rows
set sc=gc.Fetch(hstmt)
if $$$ISERR(sc) quit sc
set rownum=1
while((gc.sqlcode'=100) && (rownum<=200)) {
    for ii=1:1:numcols {
        set sc=gc.GetData(hstmt, ii, 1, .val)
        write "_val"
        if $$$ISERR(sc) break
    }
    set rownum=rownum+1
    write !
    set sc=gc.Fetch(hstmt)
    if $$$ISERR(sc) break
}

//Close cursor and then disconnect
set sc=gc.CloseCursor(hstmt)
if $$$ISERR(sc) quit sc

set sc=gc.Disconnect()
quit sc
}
```

2 Creating and Using an External Data Set

To create and use a data set that queries an external database, do the following:

1. Create an instance of %SQLGatewayConnection via the **%New()** method.
2. Call the **Connect()** method of that instance, passing arguments that specify the ODBC data source name, as well as the username and password that are needed to log in to that source, if necessary.

The **Connect()** method has the following signature:

```
method Connect(dsn, usr, pwd, timeout) as %Status
```

Here *dsn* is the DSN for the data source, *usr* is a user who can log in to that data source, *pwd* is the corresponding password, and *timeout* specifies how long to wait for a connection.

3. Create an instance of %ResultSet via the **%New()** method, providing the string argument "%DynamicQueryGW:SQLGW".

Note: This is slightly different from the argument that you use with a typical dynamic query ("%DynamicQuery:SQL").

4. Invoke the **Prepare()** method of the result set. The first argument should be a string that consists of a SQL query, the second argument should be omitted, and the third argument should be the instance of %SQLGatewayConnection.

5. Call the **Execute()** method of the result set, optionally providing any arguments in the order expected by the query. This method returns a status, which should be checked.

To use the result set, you generally examine it one row at a time. You use methods of %ResultSet to retrieve information such as the value in a given column. Typically you iterate through all the rows using **Next()**, as demonstrated in the following example:

Example

Class Member

```
ClassMethod SelectAndWrite() as %Status
{
    Set conn=##class(%SQLGatewayConnection).%New()
    Set sc=conn.Connect("AccessPlayground","", "")
    If $$$ISERR(sc) do $System.Status.DisplayError(sc) quit

    Set res=##class(%ResultSet).%New("%DynamicQueryGW:SQLGW")
    Set sc=res.Prepare("SELECT * FROM PEOPLE", conn)
    If $$$ISERR(sc) do $System.Status.DisplayError(sc) quit

    Set sc=res.Execute()
    If $$$ISERR(sc) do $System.Status.DisplayError(sc) quit

    While res.Next()
    { Write !, res.GetData(1), " ", res.GetData(2), " ", res.GetData(3)
    }
    Set sc=conn.Disconnect()
    Quit sc
}
```

For more information on %ResultSet, see the chapter “[Using Dynamic SQL](#)” in *Using InterSystems SQL*. Also see the class documentation for %ResultSet.

3 Calling ODBC Functions Directly

If %SQL.Statement does not provide enough control, you can use the %SQLGatewayConnection class to access ODBC directly. It provides a set of methods that correspond to ODBC functions (see “[Supported ODBC Function Calls](#)”), as well as other utility functions. You can connect to and use an ODBC-compliant database and then perform low-level ODBC programming. The overall procedure is as follows:

1. Create an instance of %SQLGatewayConnection via the **%New()** method.
2. Call the **Connect()** method of that instance, passing arguments that specify the ODBC data source name, as well as the username and password that are needed to log in to that source, if necessary.
3. Call the **AllocateStatement()** method and receive (by reference) a statement handle.
4. Call other methods of the SQL Gateway instance, using that statement handle as an argument. Most of these methods call ODBC functions.

The following simple example demonstrates this procedure. It is similar to the example in the previous section, but it uses the %SQLGatewayConnection versions of **Prepare()** and **Execute()** to call ODBC query functions **SQLPrepare()** and **SQLExecute()** directly, rather than using the %SQL.Statement methods:

Executing a query using %SQLGatewayConnection methods

Class Member

```
ClassMethod ExecuteQuery(mTable As %String)
{
    set mDSN="DSNtest"
    set mUserName="SYSDBA"
    set mUsrPwd="masterkey"

    // Create an instance and connect
    set gateway=##class(%SQLGatewayConnection).%New()
    set status=gateway.Connect(mDSN,mUserName,mUsrPwd)
    if $$$ISERR(status) do $System.Status.DisplayError(status) quit $$$ERROR()
    set hstmt=""

    // Allocate a statement
    set status=gateway.AllocateStatement(.hstmt)
    if $$$ISERR(status) do $System.Status.DisplayError(status) quit $$$ERROR()

    // Use %SQLGatewayConnection to call ODBC query functions directly
    set status=gateway.Prepare(hstmt,"SELECT * FROM "_mTable)
    if $$$ISERR(status) do $System.Status.DisplayError(status) quit $$$ERROR()
    set status=gateway.Execute(hstmt)

    if $$$ISERR(status) do $System.Status.DisplayError(status) quit $$$ERROR()
    quit gateway.Disconnect()
}
```

Note: Null Values and Empty Strings

When you use the methods described in this chapter, remember that InterSystems IRIS and SQL have the following important differences:

- In SQL, "" represents an empty string.
- In InterSystems IRIS, "" equals null.
- In InterSystems IRIS, \$char(0) equals an empty string.

4 Quick Reference for %SQLGatewayConnection

- [Overview of the %SQLGatewayConnection API](#)
- [%SQLGatewayConnection Methods and Properties](#)
- [Supported ODBC Function Calls](#)

4.1 Overview of the %SQLGatewayConnection API

The %SQLGatewayConnection class provides properties and methods that you can use to manage the connection to the external data source, check status information, and get information about the ODBC shared library. The methods and properties covered in this reference are listed below, organized by usage (see “[Supported ODBC Function Calls](#)” for methods not listed here):

Managing the Connection

The %SQLGatewayConnection class provides properties and methods that you can use to manage the connection to the external data source.

- **DSN** — (%String property) Data source name of the ODBC-compliant data source to which you want to connect.

- **User** — (%String property) Username to log in to the data source.
- **Password** — (%String property) Associated password
- **ConnectionHandle** — (%Binary property) The current connection handle to the ODBC-compliant data source.
- **Connect()** — Establishes a connection to a DSN.
- **GetConnection()** — Establishes a connection using configuration settings to determine the DSN, username, and password.
- **SetConnectOption()** — Invokes the ODBC function **SQLSetConnectAttr**.
- **Disconnect()** — Closes the connection.

Status and Query Methods

Most of the methods of %SQLGatewayConnection return a status, which you should check. Status information is also available via the following properties and methods:

- **sqlcode** — (%Integer property) Contains the SQL code return by the last call (if any).
- **GatewayStatus** — (%Integer property) Indicates the status of the last call.
- **GetLastSQLCode()** — Returns an SQL code for the last call if this call does not return an SQL code.
- **GatewayStatusGet()** — Returns an error code for the last call.

The following methods get rows from the result set:

- **FetchRows()** — Returns (by reference) a specified number of rows for the given connection handle.
- **GetOneRow()** — Returns (by reference) the next row for the given connection handle.

The following methods get and set the values of bound query parameters:

- **GetParameter()** — Returns (by reference) the current value of the indicated parameter.
- **SetParameter()** — Sets the value of a previously bound parameter.

Using the Shared Library

The %SQLGatewayConnection class provides properties and methods that you can call to get information about the shared library used by the ODBC SQL Gateway.

- **DLLHandle** — (%Binary property) Handle for the shared library, as currently in use. This is set when you connect.
- **DLLName** — (%String property) Name of the shared library currently in use. This is set when you connect.
- **GetGTWVersion()** — Returns the current version of the shared library.
- **GetUV()** — Returns (by reference) whether the shared library was built as Unicode. Note that this method always returns a status of \$\$\$OK.
- **UnloadDLL()** — Unloads the shared library from the process memory.

4.2 %SQLGatewayConnection Methods and Properties

This is an alphabetical listing of selected methods and properties. See “[Supported ODBC Function Calls](#)” for methods not listed here.

AllocateStatement()

Invokes ODBC function **SQLAllocHandle()** and creates the corresponding structures in the SQL Gateway.

```
method AllocateStatement(ByRef hstmt) as %Status
```

Connect()

Establishes a connection to a DSN.

```
method Connect(dsn, usr, pwd, timeout) as %Status
```

If username and password are both empty, this method calls the ODBC function **SQLDriverConnect()**. If that call is unsuccessful or username/password are specified, the method calls the ODBC function **SQLConnect()**.

To connect with an [access token](#), specify it with the **ACCESSTOKEN** parameter. For example: (cs, the connection string, contains a database name, port, and server):

```
set cs = cs _ ";ACCESSTOKEN=" _ token _";  
s gc = ##class(%SQLGatewayConnection).%New()  
s st = gc.Connect(cs, "", "")
```

If the timeout parameter is not 0, **SQLSetConnectAttr()** is first called to set **SQL_ATTR_LOGIN_TIMEOUT**.

ConnectionHandle property

%Binary property that provides the current connection handle to the ODBC-compliant data source.

Disconnect()

Closes the connection.

```
method Disconnect() as %Status
```

DLLHandle property

%Binary property that provides the handle for the shared library, as currently in use. This is set when you connect.

DLLName property

%String property that provides the name of the shared library currently in use. This is set when you connect.

DSN property

%String property that provides the data source name of the ODBC-compliant data source to which you want to connect.

FetchRows()

Returns (by reference) a specified number of rows for the given connection handle.

```
method FetchRows(hstmt, Output rlist As %List, nrows As %Integer) as %Status
```

Here *hstmt* is the connection handle, returned (by reference) from **AllocateStatement()**. Also, *rlist* is the returned list of rows; this is an InterSystems IRIS \$list. Each item in the list contains a row. If there is no data (**SQL_CODE** = 100), fetching is assumed to be successful but the return list is empty.

CAUTION: This method is primarily useful for testing, and it truncates character fields up to 120 characters so that more fields would fit in a row. Use **GetData()** instead when you need non-truncated data.

GatewayStatus property

%String property that provides the status of the last call. Status value will be one of the following:

- 0 - success
- -1 - SQL error
- -1000 - critical error

GatewayStatusGet()

Returns an error code for the last call.

```
method GatewayStatusGet() as %Integer
```

It does not initialize the error code and can be called multiple times. See the previous notes for the `GatewayStatus` property.

GetConnection()

Establishes a connection, using configuration file entries to determine the DSN, user name, and password.

```
method GetConnection(conn, timeout) as %Status
```

GetGTWVersion()

Returns the current version of the shared library.

```
method GetGTWVersion() as %Integer
```

GetLastSQLCode()

Returns an SQL code for the last call if this call does not return an SQL code (for example, if you used `SQLGetData()`).

```
method GetLastSQLCode() as %Integer
```

GetOneRow()

Returns (by reference) the next row for the given connection handle.

```
method GetOneRow(hstmt, ByRef row) as %Status
```

Here *hstmt* is the connection handle, returned (by reference) from `AllocateStatement()`. Also, *row* is the returned row, an InterSystems IRIS \$list. Each item in the list contains a field. If there is no data (`SQL_CODE = 100`), fetching is assumed to be successful but the return list is empty.

CAUTION: This method is primarily useful for testing, and it truncates character fields up to 120 characters so that more fields would fit in a row. Use `GetData()` instead when you need non-truncated data.

GetParameter()

Returns (by reference) the current value of the indicated parameter.

```
method GetParameter(hstmt, pnbr, ByRef value) as %Status
```

Here *hstmt* is the connection handle returned (by reference) from `AllocateStatement()` and *pnbr* is the ordinal number of the parameter.

GetUV()

Returns (by reference) whether the shared library was built as Unicode.

method GetUV(ByRef infoval) as %Status

Note that this method always returns a status of \$\$\$OK.

Password property

%String property that provides the associated password.

SetConnectOption()

Invokes the ODBC function **SQLSetConnectAttr()**.

method SetConnectOption(opt, val) as %Status

Only integer values are supported. Integer values for the *opt* argument may be taken from the `sql.h` and `sql.h` header files.

SetParameter()

Sets the value of a previously bound parameter.

method SetParameter(hstmt, pvalue, pnbr) as %Status

Here *hstmt* is the connection handle returned (by reference) from **AllocateStatement()**, *pvalue* is the value to use, and *pnbr* is the ordinal number of the parameter. The parameters are stored in \$list format. If the allocated buffer is not sufficient, a new buffer will be allocated.

sqlcode property

%Integer property that provides the SQL code returned by the last call (if any).

UnloadDLL()

Unloads the shared library for the ODBC SQL Gateway from the process memory.

method UnloadDLL() as %Status

User property

%String property that provides the username to log in to the data source.

4.3 Supported ODBC Function Calls

The following table lists ODBC functions directly supported by corresponding %SQLGatewayConnection methods, and links to the class documentation for those methods. See “[Calling ODBC Functions Directly](#)” for an example that calls methods to invoke ODBC functions **SQLPrepare** and **SQLExecute**.

This chapter is not intended as a detailed reference for these methods. For details on method arguments, actions, and return values, see the InterSystems Class Library reference for %SQLGatewayConnection.

Table 1: Calling ODBC Functions from %SQLGatewayConnection

ODBC Function	ObjectScript methods that call the function
SQLAllocHandle	AllocateStatement()

ODBC Function	ObjectScript methods that call the function
SQLBindParameter	BindParameter(), BindParameters()
SQLCloseCursor	CloseCursor()
SQLColAttribute	DescribeColumns()
SQLColumnPrivileges	ColumnPrivileges(), ColumnPrivilegesW()
SQLColumns	Columns(), ColumnsW()
SQLDescribeCols	DescribeColumns()
SQLDescribeParam	DescribeParameters()
SQLDiagRec	GetErrorList()
SQLEndTran	Transact()
SQLExecute	Execute()
SQLFetch	Fetch()
SQLForeignKeys	ForeignKeys(), ForeignKeysW()
SQLFreeHandle	DropStatement()
SQLFreeStmt	UnbindParameters()
SQLGetData	GetData(), GetDataL(), GetDataLW(), GetDataW()
SQLGetInfo	GetInfo()
SQLGetTypeInfo	GetTypeInfo()
SQLMoreResults	MoreResults()
SQLNumParams	DescribeParameters()
SQLParamData	ParamData()
SQLPrepare	Prepare(), PrepareW()
SQLPrimaryKeys	PrimaryKeys(), PrimaryKeysW()
SQLProcedureColumns	DescribeProcedureColumns(), DescribeProcedureColumnsW()
SQLProcedures	DescribeProcedures(), DescribeProceduresW()
SQLPutData	PutData(), PutDataW()
SQLRowCount	RowCount()
SQLSetConnectAttr	SetConnectOption()
SQLSetStmtAttr	SetStmtOption()
SQLSpecialColumns	SpecialColumns(), SpecialColumnsW()
SQLStatistics	Statistics(), StatisticsW()
SQLTablePrivileges	TablePrivileges(), TablePrivilegesW()
SQLTables	Tables(), TablesW()

